Rabbit: Retrieval-Augmented Generation Enables Better Automatic Database Knob Tuning

Wenwen Sun†, Zhicheng Pan†, Zirui Hu, Yu Liu, Chengcheng Yang‡, Rong Zhang, Xuan Zhou
School of Data Science and Engineering, East China Normal University
*Engineering Research Center of Blockchain Data Management, Ministry of Educatio*n
{wwsun, zcpan, zrhu, yliu61}@stu.ecnu.edu.cn, {ccyang, rzhang, xzhou}@dase.ecnu.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00284

***Abstract*—The large language model (LLM)-based knob tuning method has attracted considerable attention due to its excellent in-context learning ability and generalizability. However, the existing LLM-based tuning methods do not effectively harmonize multi-source external knowledge, leading to missed opportunities for enhanced knob tuning. In light of this, we propose RABBIT, a novel approach that leverages Retrieval-augmented generation to enhance database knob tuning tools, which seamlessly integrates structured historical tuning experience with graph-encoded static knowledge. First, we introduce an experience-driven knob selection strategy, enhanced by dependency-aware external knowledge integration, to systematically select key knobs. Second, we develop a cutting-edge multi-agent knob domain pruning method, which ensures the reduced search space remains compact yet effective. Finally, we leverage the few-shot capabilities of LLMs to act as surrogate models, enabling rapid exploration of the pruned search space, followed by incremental optimization that expands the search space using historical insights. Moreover, we also design an adaptive strategy to transition between these two search spaces, striking an optimal balance between exploration and exploitation. Extensive experiments on well-established benchmarks demonstrate that RABBIT outperforms the state-of-the-art methods in both effectiveness and efficiency, pointing to a new paradigm for this area.**

***Index Terms*—Knob tuning, LLM, RAG, Bayesian optimization**

I. INTRODUCTION

To effectively control the runtime behavior, DBMSs expose dozens of configurable knobs, such as *log_file_size*, *temp_buffer_size*, etc. Setting these knobs correctly for a specific workload could significantly improve the system performance. However, manually tuning these knobs is a non-trivial task, even for experienced database administrators (DBAs) since they are struggling to reason about how to tune the hundreds of knobs for each application. Therefore, researchers have proposed to solve this problem in an automated manner.

Recent studies on automated knob tuning have shown to achieve better performance compared with expert DBAs. Specifically, they can be broadly classified into three categories: ❶ the *heuristic*-based methods [1]–[4] employ hard-coded rules to explore the knob configuration space and recommend the optimal configuration [5]. These hard-coded rules highly depend on expert experience and lack generalizability across different workloads; ❷ the *reinforcement learning* (RL)-based methods [6]–[8] represent a significant advancement by making the knob recommendation process learnable and data-driven, evolving through a number of training efforts to enhance the generalization ability. Nevertheless, RL-based methods require a substantial computational overhead since they need thousands of iterations to converge [9]; ❸ the *Bayesian optimization* (BO)-based methods [10]–[13] utilize a probabilistic surrogate model to map the relation between knob configuration and DBMS performance, thus guiding the tuning process to find the optimal knob configuration by maximizing the acquisition function. Empirically, both the RL-based and BO-based methods could achieve competitive knob-tuning results. However, the BO-based methods exhibit a faster convergence speed, resulting in lower tuning time [14]–[16].

Recently, considering the excellent in-context learning ability and generalizability of large language models (LLMs), several LLM-based knob tuning methods have been proposed, e.g., GPTuner [17], LLMTune [18], LATuner [19], etc. They generally leverage LLMs to guide BO-based knob tuning for high efficiency. However, using LLMs to enhance the whole process of knob tuning has always been tough work. The key challenges of existing studies are summarized as follows:

(C1) Lack of a Versatile Knowledge Base. While LLMs demonstrate impressive general capabilities, they still lack specialized knob tuning expertise. In light of this, a knowledge base becomes essential. Unfortunately, existing efforts [18], [20] fail to structure static knowledge (e.g., database manuals) into informative formats (e.g., graphs). This omission would neglect the relationships between interdependent knobs, which are crucial for identifying synergistic configurations that help maximize the tuning performance [10], [21]. Moreover, existing approaches directly prompt LLMs with unprocessed text for specific knobs without adequately chunking [17], [19], [22], limiting their ability to aggregate semantically related fragments across multi-topic documents. As a result, this would not only increase the computational cost of large-context processing, but also limit the precision and scalability of knowledge utilization. Finally, existing studies also ignore valuable historical tuning experiences, despite "*experience is the best teacher*". While static knowledge provides general guidelines, historical tuning data offers context-specific insights. With sufficient collaborations between static knowledge and dynamic experiences, more opportunities can be found to enhance the tuning adaptability and robustness.

(C2) Ineffective Reduction of the Search Space. Re-

†Equal contribution. ‡Corresponding author.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00284

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

ducing the search space is essential for improving tuning efficiency, which involves two aspects: ❶ *key knob selection* and ❷ *domain pruning*. For ❶, existing methods [9], [11], [17], [22], [23] fail to effectively incorporate sophisticated external knowledge, which often result in a knob set that lack tuning synergy. For example, the effect of the knob *innodb_adaptive_max_sleep_delay* depends on the configuration of the knob *innodb_adaptive_sleep_delay*. Ignoring such inherent dependencies would hinder the correct selection of optimal key knobs. Furthermore, historical tuning experiences could complement external knowledge by capturing knob significance in multiple similar tasks. However, none of existing approaches combines sophisticated static knowledge with dynamic historical insights to enhance the knob selection, leading to suboptimal results. For ❷, the domain pruning [17], [23] faces similar challenges. On the one hand, the external knowledge provides the reasonable domain reference for each knob, which could effectively avoid the issue of overly aggressive pruning that excludes promising configurations. On the other hand, the historical experience could help refine the pruned domain and validate its effectiveness. Unfortunately, none of the existing methods organically integrates them, ultimately limiting potential performance improvements.

(C3) Insufficient Guidance for Configuration Optimization. Existing methods generally follow the conventional *try-collect-adjustment* tuning cycle [10]–[13], [17], [19], [22], [23], which suffers from two critical limitations. First, the effectiveness of configuration optimization requires the guidance of accurate surrogate models. However, these models struggle to achieve high accuracy due to the limited observations. Moreover, these techniques do not take advantage of the informative knob tuning knowledge. As a result, they are often misled to a local optima. Second, after the pruned search space has been sufficiently explored, there exists a requirement of transition to a broader search space that might contain better configurations. However, existing methods employ simple heuristics (e.g., fixed iterations) to determine when to perform such a transition, which could not balance the trade-off between the completeness of exploring the pruned space and the comprehensiveness of exploring the broader space.

To address these challenges, we propose RABBIT, a novel approach that leverages Retrieval-augmented generation to enhance database knob tuning tools. First, we construct a versatile knowledge base that combines graph-encoded static knowledge with structured historical tuning experiences. This allows us to incorporate external knowledge, providing precise and context-aware guidance throughout the tuning process. Second, we introduce a dependency-aware knob selection mechanism that identifies key knobs based on their inter-dependencies and historical insights. To further refine the search space, we incorporate a multi-agent domain pruning strategy, which ensures that the reduced search space remains both compact and effective. Ultimately, we design a few-shot surrogate model based on LLMs to efficiently explore the pruned search space. To ensure thorough exploration, we introduce a well-defined metric that evaluates whether the pruned space has been sufficiently searched, enabling a flexible transition to a broader search space when necessary. In this context, we propose an experience-driven incremental optimization approach to further refine the tuning process.

In summary, this paper makes the following contributions:

- We propose a novel knob tuning framework based on retrieval-augmented generation, called RABBIT, which seamlessly integrates structured historical tuning experience with graph-encoded static knowledge.
- We introduce an experience-driven knob selection strategy, enhanced by dependency-aware external knowledge integration, to systematically select key knobs.
- We develop a cutting-edge multi-agent domain pruning for knob tuning, enabling a highly compact and impactful search space tailored for efficient optimization.
- We propose an innovative two-stage configuration optimization framework that leverages LLMs' few-shot capabilities for efficient exploration, followed by incremental optimization using historical insights, striking an optimal balance between exploration and exploitation.
- Extensive evaluations on four widely-used benchmarks demonstrate that our RABBIT achieves significant improvements in both effectiveness and efficiency.

II. BACKGROUND

A. Database Knob Tuning

DBMSs expose hundreds of configurable knobs to control their runtime behaviors [5]. The DBMS knob tuning problem is to select an appropriate value for each knob to optimize the DBMS performance on a certain workload. Formally, given a set of configurable knobs $\kappa_1, \kappa_2, \ldots, \kappa_n$ along with their domains $\Theta_1, \ldots, \Theta_n$, the configuration space is defined as $\mathcal{X} = \Theta_1 \times \cdots \times \Theta_n$. We aim to find a configuration $\mathbf{x}^\star \in \mathcal{X}$ that maximizes the black-box performance function $f(\cdot)$:

$$\mathbf{x}^\star = \arg\max_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}).$$

In the literature, three methodologies are proposed as follows:

Heuristic-based. Heuristic-based methods [1], [4], [5] are divided into two categories: rule-based and search-based. 1) Rule-based techniques use predefined static rules for exploration. 2) Search-based techniques employ various heuristics to navigate the space, like refraining from revisiting areas and probing adjacent regions to enhance the current best solution.

RL-based. RL methods proposed in [6]–[8], [20] leverage the deep deterministic policy gradient (DDPG) [24], which trains an RL actor using a trial-and-error approach. The actor utilizes a deep neural network that is trained based on DBMS internal metrics, and can propose configurations that balance exploration and exploitation. In general, for RL methods to work well, thousands of samples have to be evaluated and prior work has found that RL-based methods require more iterations due to the complexity of the neural networks used.

BO-based. The majority of prior database knob tuners use BO-based methods [10]–[13], [15], [23], [25], which follow the generic BO framework to find an optimal configuration.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

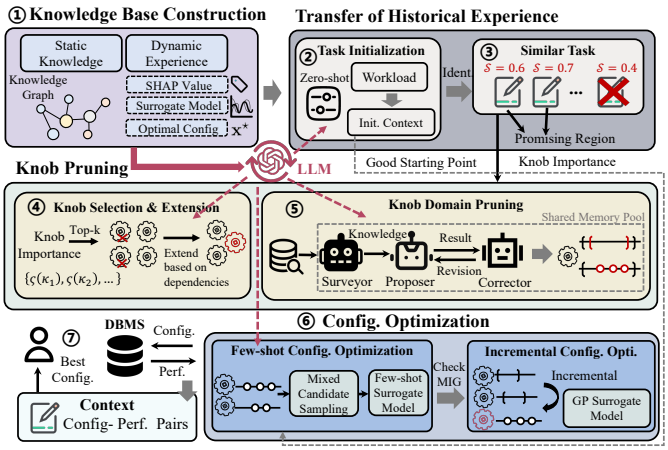


Fig. 1. System Overview of RABBIT.

Due to space constraints, we refer readers to the recent survey [26] for details, and focus on the existing methods which improve BO-based methods with LLMs. Fan et al. [19] and Zhang et al. [27] explored the opportunities of using LLM as a configuration optimizer. LLMTune [18] leverages LLM to recommend the initial configuration as a good starting point, thus enhancing efficiency. However, LLMTune requires a relatively substantial training overhead. As the state-of-the-art LLM-enhanced tuning approach, GPTuner [17] leverages online tuning documents and uses LLMs to read manuals to enhance the tuning process, posing a promising alternative.

Note, the tuning paradigm studied in this paper follows an *offline* knob tuning approach, which is also the case for previous studies [6]–[8], [10]–[13], [15], [17]–[20], [23]–[29].

B. Retrieval-Augmented Generation

LLMs have advanced significantly in applications but still have considerable limitations, especially in domain-specific and knowledge-intensive tasks [30]. A notable challenge is their tendency to produce *hallucinations* [31] when addressing queries that extend beyond their training data or require *fresh* information. To address these challenges, retrieval-augmented generation (RAG) has emerged as a valuable approach. RAG enhances LLMs by retrieving relevant document segments from external knowledge bases through semantic similarity computations. By referencing external knowledge, RAG effectively mitigates the occurrence of factually inaccurate content.

Recent works highlight the significant potential of RAG in various applications [32], [33]. *KnowledGPT* [34] generates search queries for a knowledge base and maintains a personalized knowledge repository, thereby enriching the LLMs' informational foundation. *Self-RAG* [35] refine the RAG framework by enabling LLMs to actively determine the optimal moments and content for retrieval. Recently, *Graph RAG* [36] introduced a novel graph-based text indexing approach using LLMs. It constructs an entity knowledge graph from source documents and pre-generates community summaries for related entities.

III. OVERVIEW OF RABBIT

As shown in Fig. 1, the workflow of our RABBIT consists of the following seven steps: ❶ **Knowledge base construction**: we initiate the establishment of a versatile knowledge base that integrates both static knowledge and dynamic experience. Thus, it can serve as an external knowledge guidance throughout the life-cycle of our RABBIT. ❷ **Task initialization**: we provide the meta-information of the specific DBMS (e.g., MySQL 8.0), the target workload (e.g., TPC-C), the hardware information (e.g., 4C8G) and the tuning objective (e.g., throughput). Then, RABBIT subsequently generates several diverse yet effective configurations for evaluation, aiming to characterize the target task and establish a good starting point. ❸ **Searching experience**: RABBIT identifies similar tasks by considering whether the target and historical tasks have similar performance rankings for different configurations. Once these similar tasks are obtained, the effective insights within these tasks can assist the current tuning process. ❹ **Knob selection**: RABBIT employs an historical experience driven knob selection strategy, enhanced by dependency-aware external knowledge integration, to systematically select key knobs. ❺ **Domain pruning**: although the key knobs are acquired, their domains are still vast. Thus, RABBIT reconstructs a tight and high-quality search space through a multi-agent-based knob domain pruning module. With the collaboration and adversary of multiple agents, RABBIT effectively reduces the continuous domain to multiple promising discrete candidate values. ❻ **Configuration optimization**: RABBIT employs a two-stage configuration optimization. In the first stage, it leverages a few-shot surrogate model to rapidly explore the pruned search space. In the second stage, it performs an incremental optimization by gradually expanding the explored search space according to historical insights. In addition, the *information gain* metric is used to adaptively transit between the two stages. ❼ **Recommendation**: RABBIT returns the best-performing configuration to the user.

IV. KNOB KNOWLEDGE BASE CONSTRUCTION

As discussed in §I, knob tuning can benefit significantly from leveraging both static knowledge (e.g., database manuals) and dynamic experience (e.g., historical tuning tasks) as *auxiliary information sources*. Therefore, we propose constructing a versatile knowledge base that integrates these information sources, thereby providing robust guidance for tuning.

The Necessity of LLMs. However, this integration requires methods that can conduct deep semantic analysis and contextual reasoning. Without LLMs, the utilization of the knowledge base would be severely constrained. Although a database administrator (DBA) could manually craft white-box rules through the knowledge base, it is a labor-intensive process [15], prone to human error and impractical for relational databases. Moreover, heuristic retrieval from the knowledge base offers an alternative method by querying a knowledge graph to identify knob dependencies. However, this method can only handle structured and non-textual data, leaving the rich textual insights of the knowledge base underutilized.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

Fortunately, LLMs excel at processing and synthesizing vast amounts of textual information by leveraging advanced natural language understanding and contextual reasoning. They can extract implicit relationships, generate insights from unstructured data, and bridge the tuning knowledge gaps.

Knowledge Representation. The existing method [17] proposes *reading* the database manual to infer an optimized search space for each knob. However, it has two key limitations. First, it overlooks the inter-dependencies between knobs. Nevertheless, jointly tuning multiple interrelated knobs can yield greater overall performance improvements [10]. Second, it prompts the LLMs with the original knowledge text for specific knobs without chunking, which prevents finer-grained searches and limits the integration of semantically relevant chunks. This not only limits the scalability of the knowledge base, but also increases the risk of excessive large-context loading and ineffective handling of multi-topic documents. Beyond these issues, historical experience is similar to a *program log* without explicit semantics, making them difficult to interpret directly. Therefore, an additional challenge lies in regularizing these logs to ensure they can be effectively utilized.

To address these issues, we propose encapsulating static knowledge into a knowledge graph to fully utilize its information (see §IV-A), and structuring dynamic experience for effective reuse, e.g., transfer from similar tasks (see §IV-B).

A. Static Knowledge

As the knowledge graph technology has been well-recognized to capture knowledge associations effectively [37], [38], we aim to encapsulate the static knowledge into a comprehensive knob knowledge graph $\mathcal{G}_{knob}$. Next, we discuss how to construct $\mathcal{G}_{knob}$ step by step.

❶ **Chunking.** We first divide the textual tuning corpus (e.g., database manuals) into multiple chunks. These chunks are the smallest analyzable units, with a certain number of tokens (e.g., 1,200) as the splitting principle. By segmenting long documents into smaller chunks, it is easier for us to aggregate semantically related chunks across multi-topic documents.

❷ **Element Extraction.** We then use the split chunks to prompt LLMs to identify and extract all entities (e.g., knob name, description, default suggestions, etc.), and the relationships between them. Then, we can use this extracted information to construct an initial knowledge graph $\mathcal{G}_{knob}$.

❸ **Hierarchical Clustering.** We further employ the *hierarchical Leiden algorithm* [39] to effectively discover community structures within the initial knowledge graph. Note, *community* is a group of nodes within the graph that are densely connected to each other but sparsely connected to other dense groups in the network.

❹ **Community Summary.** We generate summaries for each community member and then aggregate them using a bottom-up approach to obtain the community summary. This allows us to effectively summarize main entities and their relationships within the community. The entire textual tuning corpus is then roughly divided into community summaries, highlighting the collaborative effectiveness of knobs.

❺ **Semantic Vectorization.** Although we have constructed an initial knowledge graph $\mathcal{G}_{knob}$, it is *explicit*, which could not handle scenarios that require semantic understanding. Therefore, we leverage OpenAI's embedding API to *implicitly* embed the aforementioned elements in $\mathcal{G}_{knob}$, enabling a more flexible retrieval, especially in cases where relevant information is semantically similar but not explicitly linked.

B. Dynamic Experience

Relying solely on static knowledge is insufficient, as different workloads and environments might require distinctive optimal configurations. To address this issue, we incorporate insights from historical experiences. Specifically, we define a *tuning task* as the process of evaluating multiple configurations to optimize performance. The representation of these tasks, a.k.a., *experience*, has been widely recognized as effective in improving target task tuning through *transfer learning* [25], [40], [41]. To concisely yet comprehensively summarize a tuning task, we consider three key features: ❶ SHAP values, ❷ surrogate models, and ❸ optimal configurations.

❶ **SHAP Value.** We consider distinguishing the contributions of different knobs in a task. Due to the numerous observations in the tuning process, SHapley Additive exPlanation (SHAP) [42] could be utilized to handle this problem, which is well-established for evaluating feature importance by analyzing performance variations across different configurations.

More specifically, for any knob κ_i in a task τ, the SHAP value $\varrho(\kappa_i)$ represents its contribution to the overall tuning effect across all possible combinations with other knobs:

$$\varrho(\kappa_i) = \sum_{s} \frac{|s|!(n-|s|-1)!}{n!} \left(v\left(s \cup \{\kappa_i\}\right) - v(s)\right), \tag{1}$$

where s is the subset of knobs used in the model, denoted as $s \subseteq \{\kappa_1, \kappa_2, \ldots, \kappa_n\} \setminus \{\kappa_i\}$, n is the number of the adjusted knobs in the task, $v\left(s \cup \{\kappa_i\}\right) - v(s)$ denotes the average performances across all possible values of other knobs when fixing κ_i. Consequently, we derive a SHAP value for each knob, represented as $\boldsymbol{\varrho} = \langle \varrho(\kappa_1), \varrho(\kappa_2), \ldots, \varrho(\kappa_n) \rangle$.

❷ **Surrogate Model.** Given a tuning task consisting of configuration-performance pairs $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots\}$, it becomes feasible to construct a surrogate model $\tilde{f}_\theta$ that approximates the intrinsic configuration-performance relationship. Consequently, we can utilize $\tilde{f}_\theta$ to forecast the performance of any configuration, namely, $\tilde{y} = \tilde{f}_\theta(\mathbf{x})$.

❸ **Optimal Configuration.** We also store the optimal configuration $\mathbf{x}^\star$. This is because $\mathbf{x}^\star$ is crucial and it is likely to be an effective candidate for the target task $\hat{\tau}$.

Finally, the key features of a specific tuning task can be represented as $\boldsymbol{\varepsilon} = (\boldsymbol{\varrho}, \tilde{f}_\theta, \mathbf{x}^\star)$. Here, $\boldsymbol{\varrho}$ represents the list of SHAP values of knobs, the $\tilde{f}_\theta$ represents the trained surrogate model, and $\mathbf{x}^\star$ represents the optimal configuration.

V. Transfer of Historical Experience

A. Target Task Initialization

Task initialization refers to the use of several configuration-performance pairs to preliminarily characterize the target task.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

An effective task initialization should satisfy two key requirements. First, it should accurately characterize the properties of the current task to enable efficient identification of similar tasks. Second, it should provide a *good starting point* that enhances the convergence of the subsequent tuning process.

To this end, we employ a *zero-shot initialization strategy* to efficiently sample configurations from the search space $\mathcal{X}$. Specifically, we prompt the LLM with DBMS information, hardware information, workload information and knob information, to obtain m initial configurations, $\{\mathbf{x}_i\}_{i=1}^{m}$, where m is set to 10 [17], [19], [25], [27]. Compared to traditional methods, e.g., *random sampling*, this strategy leverages contextual understanding of LLMs to produce *reliable* and *diverse* configurations tailored to the task, mitigating randomness.

The sampled configurations are actually evaluated to form an initial context set, which plays a pivotal role in identifying similar tasks, as it captures the interaction between the sampled configurations and the underlying task environment. Note, the optimal configuration-performance pair will serve as the starting point for the subsequent tuning process (see §VII).

B. Similar Task Identification

Task Similarity. For the target task $\hat{\tau}$, we first find its similar tasks. Specifically, we evaluate the similarity between the target task $\hat{\tau}$ and each historical task τ_i by considering whether the two tasks have similar performance rankings for different configurations. Given two observations $(\mathbf{x}_1, y_1)$ and $(\mathbf{x}_2, y_2)$ during the target task $\hat{\tau}$'s initialization phase (see §V-A). If the surrogate model $\tilde{f}_\theta$ for τ_i accurately predicts their relative outcomes (i.e., $y_1 < y_2$ and $\tilde{f}_\theta(\mathbf{x}_1) < \tilde{f}_\theta(\mathbf{x}_2)$), this indicates a *consistent partial pair* between $\hat{\tau}$ and τ_i. The proportion of these pairs reflects the generalization ability of τ_i to $\hat{\tau}$, termed *similarity*, is represented as $\mathcal{S}(\tau_i, \hat{\tau})$:

$$\begin{aligned} \mathcal{S}(\tau_i, \hat{\tau}) &= \frac{2}{|\mathcal{I}|\left(|\mathcal{I}|-1\right)} \mathcal{P}(\tau_i, \hat{\tau}), \\ \mathcal{P}(\tau_i, \hat{\tau}) &= \sum_{j=1}^{|\mathcal{I}|} \sum_{k=j+1}^{|\mathcal{I}|} \mathbb{1}\left(y_j \leq y_k\right) \otimes \mathbb{1}\left(\tilde{f}_\theta\left(\mathbf{x}_j\right) \leq \tilde{f}_\theta\left(\mathbf{x}_k\right)\right), \end{aligned} \tag{2}$$

where $\mathcal{I}$ denotes the context set from §V-A, $\mathcal{P}(\tau_i, \hat{\tau})$ represents the number of consistent partial pairs, and $\otimes$ is the exclusive-nor operator. We divide $\mathcal{P}(\tau_i, \hat{\tau})$ by the number of all possible partial pairs and scale it to $[0, 1]$. Then, a higher ratio indicates a greater similarity between $\hat{\tau}$ and the historical task τ_i.

Then, we employ a threshold-based similarity query strategy to identify relevant tasks. In practice, we use a widely applied threshold (0.6) to conduct a conservative *transfer learning* [43] and avoid negative transfer caused by dissimilar tasks.

C. Knob Importance and Promising Region

Importance. Given H similar historical tasks $\mathcal{H} = \langle \tau_1, \tau_2, \ldots, \tau_H \rangle$ of the target task $\hat{\tau}$, we prefer the knobs that have made significant contributions in multiple similar tasks. Then, for any knob κ, we propose using its *relative importance* in each of $\hat{\tau}$'s similar tasks to determine its importance to $\hat{\tau}$. The higher the importance of the knob κ (denoted as $\varsigma(\kappa)$), the higher the potential benefit of tuning κ for $\hat{\tau}$.

Relative importance. Given a similar task τ_i, we first use its historical experience to obtain the SHAP values of all knobs. The SHAP value of a knob represents its specific contribution to the tuning task τ_i. Since τ_i and $\hat{\tau}$ share similar characteristics, these SHAP values also indicate the potential impact of each knob on $\hat{\tau}$. To obtain the *relative importance* $\tilde{\varsigma}_i(\kappa)$, we adjust all $\varrho_i(\kappa)$ values to be non-negative using a normalization technique, represented as:

$$\tilde{\varsigma}_i(\kappa) = \frac{\varrho_i(\kappa) - \min_{\kappa_q \in \boldsymbol{\kappa}_{all}} \varrho_i(\kappa_q)}{\sum_{\kappa_p \in \boldsymbol{\kappa}_{all}} \left(\varrho_i(\kappa_p) - \min_{\kappa_q \in \boldsymbol{\kappa}_{all}} \varrho_i(\kappa_q)\right)}, \tag{3}$$

where $\min_{\kappa_q \in \boldsymbol{\kappa}_{all}} \varrho_i(\kappa_q)$ denotes the minimum value of $\varrho_i(\kappa)$. This ensures that all normalized values are within the range $[0, 1]$, making them comparable across different τ_i.

Final importance. Due to the fact that the importance of a specific knob κ might vary across various similar tasks, we propose aggregating their relative importance (i.e., $\tilde{\varsigma}_1(\kappa), \cdots, \tilde{\varsigma}_H(\kappa)$) to generate the final importance. Furthermore, the importance rankings of the knobs are more reliable in tasks with greater similarities. Therefore, we propose a weighted sum of relative importance according to the similarity between the historical tasks τ_i and the target task $\hat{\tau}$, obtaining the final knob importance $\varsigma(\kappa)$ as follows:

$$\varsigma(\kappa) = \textstyle\sum_{\tau_i \in \mathcal{H}} \mathcal{S}(\tau_i, \hat{\tau}) \cdot \tilde{\varsigma}_i(\kappa). \tag{4}$$

Promising Region Collection. Additionally, we extract the optimal configuration $\mathbf{x}_i^\star$ from each similar tuning task, and regard it as a promising candidate. Then, we integrate and structure all the promising candidates into a promising range set $\mathbf{R}$ and a promising value set $\mathbf{V}$. Specifically, for each knob κ_j, $V_j \in \mathbf{V}$ is the set containing all of κ_j's promising configurations (i.e., the optimal values in different similar tasks), and the range $R_j \in \mathbf{R}$ is composed of the minimum and maximum values in V_j.

VI. Knob Pruning with External Knowledge

As discussed in §II-A, our objective is to tune the knobs $\kappa_1, \kappa_2, \ldots, \kappa_n$ within their respective domains $\Theta_1, \ldots, \Theta_n$. This would create a search space $\mathcal{X} = \Theta_1 \times \cdots \times \Theta_n$. However, the vast space of $\mathcal{X}$ poses significant challenges on the efficiency and effectiveness of the tuning process. Therefore, we introduce the *knob pruning* to transform $\mathcal{X}$ to $\hat{\mathcal{X}}$, where $\hat{\mathcal{X}}$ is a more compact search space.

Our knob pruning consists of two stages. The first stage *knob selection* $\mathcal{F}_{ks}(\cdot)$ involves selecting a key knob set $\boldsymbol{\kappa}_{key}$, i.e., $\boldsymbol{\kappa}_{key} \leftarrow \mathcal{F}_{ks}(\boldsymbol{\kappa}_{all})$ (see §VI-A) . The second stage *domain pruning* $\mathcal{F}_{dp}(\cdot)$ involves finely adjusting the permissible range (i.e., domain) of these knobs (see §VI-B). For example, for a knob κ along with its domain $\Theta(\kappa)$, we can obtain $\hat{\Theta}(\kappa) \leftarrow \mathcal{F}_{dp}(\Theta(\kappa))$. Formally, we can get $\hat{\mathcal{X}} = \underset{\kappa_i \in \boldsymbol{\kappa}_{key}}{\times} \mathcal{F}_{dp}(\Theta(\kappa_i))$.

A. Knob Selection & Extension

Recent studies [7], [10], [11], [17], [23], [44] have demonstrated that tuning a selected subset of knobs $\boldsymbol{\kappa}_{key} \subset \boldsymbol{\kappa}_{all}$, a.k.a., *key* knobs, could significantly enhance performance. This improvement can be attributed to two reasons [14]. First,

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

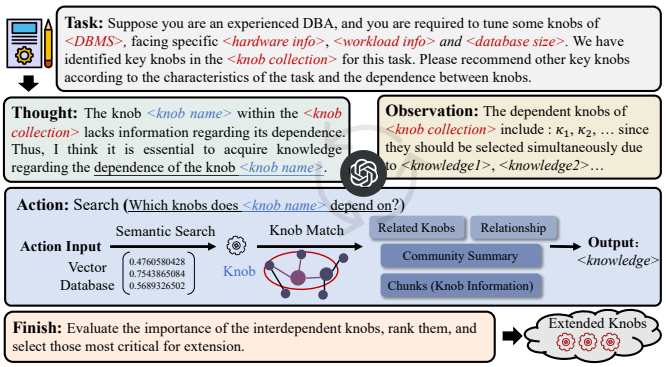


Fig. 2. Thought-Action-Observation (T-A-O) Workflow of Knob Extension.

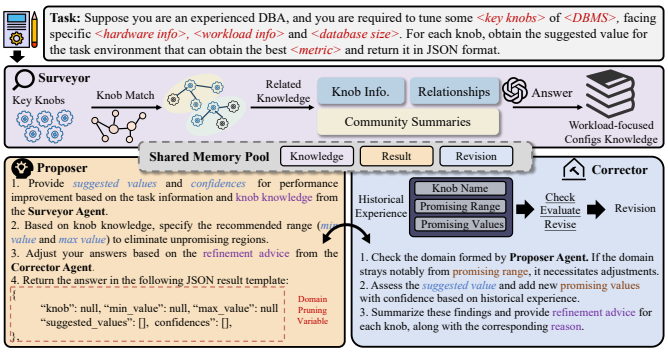


Fig. 3. Multi-Agent-based Domain Pruning (MADP).

it reduces the search space for more efficient exploration. Second, it mitigates the risk of noise from suboptimal configurations, which could negatively affect overall performance.

To identify the key knobs $\boldsymbol{\kappa}_{key}$ for the target task, existing methods [7], [10], [11], [23] measure the significance of different knobs through numerous evaluations, resulting in a huge overhead and ignoring the dependence between knobs. In light of this, we first propose using experience to obtain an initial knob set. Then, we further extend this set according to the dependence between knobs in the knowledge graph $\mathcal{G}_{knob}$.

Knob Selection. We propose selecting knobs according to their importance in historical tasks that are similar to the target task $\hat{\tau}$. To this end, we rank knobs based on their final importance (see Eq. 4), and select the k knobs with the highest importance values. Although these initially selected knobs could serve as promising candidates, the knob dependency might be lost when using Eq. 4 to aggregate knob importance values from different tasks. However, it has been proven that tuning interdependent knobs simultaneously could help improve the performance [10], [17]. Therefore, we further leverage the knob knowledge graph $\mathcal{G}_{knob}$ obtained in §IV-A to extend the $\boldsymbol{\kappa}_{base}$ with interdependent knobs $\boldsymbol{\kappa}_{add}$, thereby obtaining the complete key knobs $\boldsymbol{\kappa}_{key} = \boldsymbol{\kappa}_{base} \cup \boldsymbol{\kappa}_{add}$.

Knob Extension. Directly prompting the LLM to extend the key knobs might lead to hallucination issues, resulting in the extended knobs with weak dependencies or the omission of critical knobs. Therefore, as illustrated in Fig. 2, we employ a Chain of Thought (CoT) prompt based on ReAct [45] to progressively extend the knob set $\boldsymbol{\kappa}_{base}$, following an iterative process of Thought-Action-Observation (T-A-O).

More specifically, ❶ **(T)**: Given the knob set $\boldsymbol{\kappa}_{base}$, the LLM combines the current observation (e.g., currently acquired insights) to identify knowledge gaps that hinder the comprehensive extension of $\boldsymbol{\kappa}_{base}$. For example, the LLM would conclude that "it is essential to acquire knowledge regarding the dependency of the knob *innodb_spin_wait_delay*". Then, it is synthesized into a well-defined query for subsequent knowledge retrieval. ❷ **(A)**: According to the result of the previous thought, we search the semantic vectorization database (see §IV-A) and obtain the relevant knob embeddings. These embeddings are further used to derive their corresponding explicit entities within the $\mathcal{G}_{knob}$. Finally, the related information for these knobs (including interdependent knobs, relationships, community summaries, and original knob details) retrieved in $\mathcal{G}_{knob}$ is provided as input for the subsequent phase. ❸ **(O)**: combined with the related information from the previous step, the LLM summarizes the observations obtained in the current iteration. For example, "the dependent knobs of $\boldsymbol{\kappa}_{base}$ include κ_5 and κ_8, since they should be selected simultaneously due to ...". Then, this observation is further handed over to the ❶ **(T)** for thinking, thus forming an iteration.

Finally, through the iterative cycles of T-A-O, the LLM evaluates the importance of the interdependent knobs. Then, the LLM ranks them and selects the most critical knobs for extension, thus forming a final key knob set $\boldsymbol{\kappa}_{key} = \boldsymbol{\kappa}_{base} \cup \boldsymbol{\kappa}_{add}$.

B. Knob Domain Pruning

Previous works [17], [23], [25] often either discretize continuous knob domains into a limited set of values or apply projection-based transformations to reduce the domain size. However, these methods inevitably risks the exclusion of high-potential candidates as they are usually aggressive to some extent. This restriction might confine the search to suboptimal regions, thereby hindering performance optimization. Striking a balance between reducing the search space and preserving high-quality candidate configurations is particularly challenging. To address this issue, we propose a multi-agent domain pruning (MADP) method, designed to align with these considerations and enhance the effectiveness of tuning process.

MADP. As shown in Fig. 3, we build three individual agents with different responsibilities, which are surveyor $\mathbf{A}^s$, proposer $\mathbf{A}^p$, and corrector $\mathbf{A}^c$ (descriptions are detailed below). By communicating through a shared memory pool [46], these agents collaboratively construct a refined search space $\hat{\mathcal{X}}$. Next, for ease of presentation, we focus on discussing the domain pruning of a single knob κ in the key knob set $\boldsymbol{\kappa}_{key}$.

More specifically, during this multi-agent collaboration, a *domain pruning variable* $\Omega(\kappa)$ consisting of four kinds of critical information is updated. It includes ❶ suggested value $\xi(\kappa)$, ❷ minimum value $l(\kappa)$, ❸ maximum value $u(\kappa)$, and ❹ confidence score $\rho(\kappa)$. After the MADP finishes, Ω is applied to construct the final domain $\hat{\Theta}(\kappa) \leftarrow \Theta(\kappa)$ for the key knob $\kappa \in \kappa_{key}$, thereby forming the refined search space $\hat{\mathcal{X}}$.

These three agents iteratively update the domain pruning variable Ω. In the i-th iteration, the surveyor $\mathbf{A}^s$ leverages

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

extensive knob knowledge to generate detailed reports that summarize relevant insights. Then, the proposer $\mathbf{A}^p$ takes into consideration the $\mathbf{A}^s$'s reports, the previous result Ω_{i-1}, with the corrector $\mathbf{A}^c$'s refinement advice, producing the result Ω_i. Note, in the first iteration, the $\mathbf{A}^p$ solely relies on the $\mathbf{A}^s$'s input. The corrector $\mathbf{A}^c$, equipped with historical experience $\boldsymbol{\varepsilon}$, performs case-based reasoning about the Ω_i and proposes refinement advice. The respective responsibilities and collaboration of the three agents are detailed as follows:

Surveyor. As shown in Fig. 3, the surveyor $\mathbf{A}^s$ is designed to provide comprehensive insights regarding the knob domain by leveraging the $\mathcal{G}_{knob}$ (see §IV-A). Specifically, we adopt a retrieval method similar to that in §VI-A. The difference is that, since the knob set $\boldsymbol{\kappa}_{key}$ have been identified, we can explicitly retrieve $\mathcal{G}_{knob}$ based on their knob names. After we have obtained the related knowledge (e.g., knob information, relationships and community summaries) that contains the domain tutorial and relevance for the knob set $\boldsymbol{\kappa}_{key}$, $\mathbf{A}^s$ leverages it to generate a holistic knob domain report.

Proposer. The proposer $\mathbf{A}^p$ is responsible for generating $\Omega = \{\xi, l, u, \rho\}$. Note, for ease of presentation, hereinafter we omit κ in the equations. Specifically, it combines hardware environment and workload information to reduce the risk of generating domain that exceed the machine's capacity and potentially cause system crashes [17], [21]. However, the domain generated by LLMs might be unsatisfactory due to the hallucination issue. Therefore, we explore a *peer-reflection* mechanism [46] that dynamically refines the previously generated suggestion based on the feedback from other agents (i.e., $\mathbf{A}^s$ and $\mathbf{A}^c$) via multiple iterations. Thus, $\mathbf{A}^p$ will incorporate knob domain knowledge retrieved by $\mathbf{A}^s$ and integrate feedback from $\mathbf{A}^c$ based on the previous output Ω_{i-1} (assume this is the i-th iteration), to make the fine-grained adjustments to the new Ω_i. This process will continuously refine the domain (reflected by the Ω), reducing hallucinations while improving the interpretability of recommendations.

Corrector. The corrector $\mathbf{A}^c$ cross-checks $\mathbf{A}^p$'s output with proven past tasks, offering refinement feedback. Initially, $\mathbf{A}^p$ generates a relatively conservative domain. However, the promising value set $\mathbf{V}$ (see §V-C) show promise for the target task and could help improve results. Thus, they should be incorporated into the domain. Moreover, the promising range set $\mathbf{R}$ offers a straightforward way to validate output values. If the domain produced by $\mathbf{A}^p$ strays notably from $\mathbf{R}$, it necessitates an adjustment of Ω.

Domain Construction. Next, we discuss how to construct the pruned domain based on $\Omega = \{\xi, l, u, \rho\}$. On the one hand, directly constructing the continuous domain using l and u is still challenging, as this continuous range is too vast to efficiently explore. On the other hand, merely relying on suggested values ξ is insufficient as it might lack diversity, potentially missing better configurations. Fortunately, we have obtained a confidence score ρ, which could reflect the certainty level of the LLM's suggestion regarding ξ as a suitable candidate. Thus, we propose constructing an effective domain with an appropriate size based on ρ. More specifically, when the confidence score ρ is high, we propose extending a relatively few number of values around the suggested value ξ. This is because a high ρ implies a greater degree of trust in ξ and allows us to focus on a narrower region. In contrast, when ρ is low, a larger number of values around ξ are extended. This is because a low ρ suggests a high uncertainty in ξ, and exploring a broader range is necessary to mitigate the risk of missing better configurations.

To this end, we propose using the confidence $\rho \in [0, 1)$ to determine the number of extended points as $n = \left\lfloor n_{\min} \cdot (n_{\max}/n_{\min})^{(1-\rho)} \right\rfloor$. Here, $n_{\min}$ and $n_{\max}$ represent the minimum and maximum number of points to extend in a single direction, respectively. The value of n smoothly following an exponential decay as ρ increases. In our experiments, setting $n_{\min} = 2$ and $n_{\max} = 5$ achieved optimal results, balancing computational efficiency and robustness. After determining n, we generate points uniformly within the ranges $[l, \xi]$ and $[\xi, u]$, resulting in $2n$ extended points in total. Formally, the set of generated points can be defined as:

$$\boldsymbol{\xi_s} = \{\xi + \frac{i}{n} \cdot (1-\rho) \cdot (b - \xi) \mid i \in \{1, \ldots, n\}, b \in \{l, u\}\}. \quad (5)$$

The intuition is to deviate the suggested value ξ in different directions (smaller or bigger) with different extents. Finally, we construct the knob domain by combining the extended points $\boldsymbol{\xi_s}$ with the suggested value ξ and two boundary values (i.e., l and u). That is, $\hat{\Theta} = \{l\} \cup \{\xi\} \cup \boldsymbol{\xi_s} \cup \{u\}$.

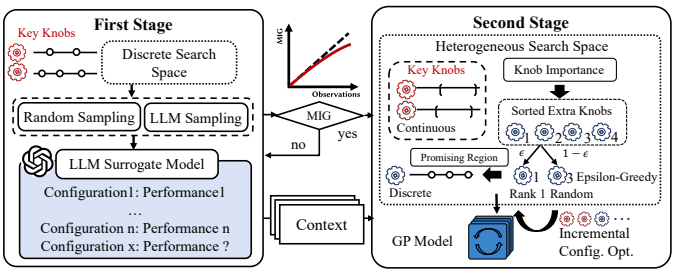


Fig. 4. Two-stage Configuration Optimization.

VII. Configuration Optimization

The *configuration optimization* process could be viewed as a *try-collect-adjust* cycle, which involves recommending knob configurations, observing their actual performance, and subsequently selecting next configuration. Traditional methods [10]–[13], [17], [23] suffer from the local optima issue due to the limited number of observation data. To address this issue, we propose a two-stage configuration optimization that fully balances exploration and exploitation, as shown in Fig. 4.

A. Few-shot Configuration Optimization

The key to Bayesian optimization (BO) lies in probabilistic sampling and selecting the next point [10]–[13], [15], [23], [25], achieving satisfactory configurations iteratively. In light of this, we consider the following three questions: ❶ how to design a surrogate model with high accuracy under limited observations; ❷ how to sample high-quality candidate configurations while exploring unknown candidates; and ❸ how to measure the exploration degree of the pruned search space.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

To address these concerns, we first allow the LLM to act as a few-shot surrogate model based on its pre-trained knowledge, even with limited observations. Then, we consider a mixed candidate-configuration sampling mechanism. On the one hand, we propose obtaining high-potential candidate configurations by prompting LLMs with external knowledge. On the other hand, the random sampling method is utilized to generate some supplementary candidate configurations, such that some unknown regions can be explored. Finally, we introduce a metric of *information gain* to measure whether the search space has been thoroughly explored. This metric can be used to determine whether we should further expand the search space (entering the second stage, see §VII-B).

❶ **Few-shot Surrogate Model.** Recent studies [47]–[51] have highlighted the remarkable capability of LLMs to offer uncertainty levels during predictions, particularly in few-shot scenarios. The advantages stem from the following factors: (1) LLMs are pre-trained on vast amounts of web-based knowledge and exhibit strong generalization capabilities. When provided with task-specific data and a few examples, they can effectively emulate surrogate models by leveraging contextual understanding. (2) Traditional surrogate models often suffer from inaccuracies due to sparse and insufficient observations, leading to suboptimal approximations. In contrast, LLMs possess a degree of database expertise, and when supplied with comprehensive environmental descriptions, they can generate more reliable and well-informed simulations. Therefore, we employ the LLM as the surrogate model $\tilde{f_\Phi}$. That is, it can predict $p\left(\tilde{y} \mid \tilde{\mathbf{x}}; \mathcal{C}\right)$, which represents the conditional probability of the predicted performance $\tilde{y}$ given a sampled candidate configuration $\tilde{\mathbf{x}}$ and the context set $\mathcal{C}$.

As shown in Fig. 4, we serialize the existing configuration-performance pairs (i.e., $\mathcal{C}$) and the specific question, e.g., "*predict the performance of a given configuration and provide the confidence level*" as the input of the LLM. Then, given any configuration $\tilde{\mathbf{x}}$, the LLM can estimate its performance $\tilde{y}$ and confidence $\tilde{\rho}$. Moreover, to obtain probabilistic estimates to form a reliable prediction distribution and mitigate LLM hallucination, this prediction is iterated t times, allowing the computation of empirical mean μ and standard deviation σ:

$$\mu = (\textstyle\sum_1^t \tilde{\rho}_i \cdot \tilde{y}_i)/\sum_1^t \tilde{\rho}_i, \quad \sigma = \sqrt{\textstyle\sum_1^t \tilde{\rho}_i (\tilde{y}_i - \mu)^2 / \sum_1^t \tilde{\rho}_i} \tag{6}$$

where $\tilde{\rho}_i$ and $\tilde{y}_i$ denote the confidence and performance value of the i-th prediction, respectively.

In addition, existing studies [22], [52], [53] have proven that the sensitivity to the ordering of in-context examples might lead to less accurate uncertainty estimates. As LLMs process inputs in a left-to-right manner, the predictions are sensitive to permutations within the prompt [22]. To further enhance the robustness of our approach, we introduce a random permutation mechanism that shuffles the few-shot examples randomly.

❷ **Mixed Candidate Sampling.** We leverage external knowledge to prompt LLMs for generating a set of high-quality candidate configurations, denoted as $\mathcal{B}_1$. These configurations are likely concentrated in good regions due to the LLM's informed generation process. However, this might overlook the unexplored regions in the search space. Therefore, we complement $\mathcal{B}_1$ with a randomly sampled set of configurations, $\mathcal{B}_2$, ensuring a broader coverage of the search space.

Next, we utilize the few-shot surrogate model (denoted as $\tilde{f_\Phi}(\cdot)$) to predict the performance of all candidate configurations in $\mathcal{B} = \mathcal{B}_1 \cup \mathcal{B}_2$. The surrogate model provides both the predicted mean $\mu(\mathbf{x})$ and the uncertainty $\sigma(\mathbf{x})$ for each candidate $\mathbf{x} \in \mathcal{B}$. Based on these predictions, we apply an acquisition function (e.g., expected improvement (EI) [10], [11]) to select the next configuration for evaluation.

❸ **Exploration Measure.** Once our tuning method has fully explored the pruned search space, we should transition to the second stage that expands the search space for broader configuration exploration. To assess whether further improvement is possible in the current search space, we use *maximal information gain* (MIG) as an indicator. Formally, we define the *information gain* (IG) based on the predictive distribution. Let $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ denote the observed configurations, and their actual performance is $\{y_1, y_2, \ldots, y_n\}$. Then, let $\mathcal{L}(\mathbf{X})$ represent the predicted likelihood of these configurations:

$$\mathcal{L}(\mathbf{X}) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(y_i - \mu_i)^2}{2\sigma_i^2}\right), \tag{7}$$

where we assume the predictions follow a Gaussian distribution. Thus, the *information gain* (IG) for $\mathbf{X}$ is defined as:

$$I(\mathbf{X}) = \mathbb{H}(\mathcal{L}_{\text{prior}}) - \mathbb{H}(\mathcal{L}_{\text{posterior}}), \tag{8}$$

where $\mathbb{H}(\cdot)$ denotes the entropy of the likelihood distribution, $\mathcal{L}_{\text{prior}}$ and $\mathcal{L}_{\text{posterior}}$ represent the prior and posterior likelihoods, respectively. Intuitively, IG measures the reduction in uncertainty regarding the surrogate model $\tilde{f_\Phi}(\cdot)$ after incorporating observations from $\mathbf{X}$. Then, the MIG is defined as the largest possible information gain over all candidate configurations:

$$\gamma_n = \max_{\mathbf{X}_n \subset \hat{\mathcal{X}}} I(\mathbf{X}_n), \tag{9}$$

where $\hat{\mathcal{X}}$ represents the entire pruned configuration space.

When MIG increases linearly with observations, it indicates the high uncertainty in the LLM's predictions, suggesting insufficient exploration. Conversely, a small MIG growth implies that $\tilde{f_\Phi}$ has sufficiently learned the objective function. Thus, in each iteration, we calculate the MIG for the sampled candidate configurations. If the MIG does not increase for three consecutive iterations, we conclude that further exploration of the current search space is unlikely to yield significant improvements. At this point, we transition to the next stage.

B. Incremental Configuration Optimization

Employing any space-reduction method would unavoidably lead to the miss of certain vital configurations [10]. In this stage, we aim to investigate an expansive space until resource exhaustion or desired performance enhancement is reached. Specifically, we identify that the previous stage has emphasized the exploration within a pruned search space, which might neglect potential improvements. Therefore, we

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

convert the discrete domain of knobs $\left\{\hat{\Theta}(\kappa)|\kappa \in \boldsymbol{\kappa}_{key}\right\}$ into a continuous domain $\{[l(\kappa), u(\kappa)]|\kappa \in \boldsymbol{\kappa}_{key}\}$. This enables a more comprehensive exploration of the extensive regions.

Leveraging the extensive observations generated in the previous stage, the traditional surrogate model (e.g., Gaussian Process, GP) [10], [11], [54] is sufficient to accurately capture the configuration-performance relationship and determine the next optimal sampling point. Compared to solely using LLMs as a surrogate model, this approach is more cost-effective and dependable due to LLMs' limitations in processing long input context, which would lead to incoherent outputs.

Incremental Optimization. Similarly, the knob selection module $\boldsymbol{\kappa}_{key} \leftarrow \mathcal{F}_{ks}(\boldsymbol{\kappa}_{all})$ (see §VI-A) might also hinder to maximize system performance. Thus, inspired by [10], we propose an incremental strategy. The difference is that we fully utilize promising regions from historical experience, since they contain the potentially optimal configurations.

In each iteration, a knob $\kappa \in \boldsymbol{\kappa}_{all} \setminus \boldsymbol{\kappa}_{key}$ is selected and added to the tuning process. However, choosing the right knob to add is vital, as knobs introduced earlier in the process are optimized more thoroughly (i.e., they have more participation in the tuning). Given our existing historical experience, we can leverage the importance $\varsigma(\kappa)$ (see §V-C) to guide this decision.

Priority. To effectively balance exploration and exploitation, the well-recognized ϵ-greedy strategy [55] is utilized. Specifically, with a probability of ϵ, we select the highest ranked knob from the remaining knobs, exploiting the current best choice. In contrast, with a probability of $1-\epsilon$, a knob is randomly selected from the entire ranking, enabling exploration of potentially promising alternatives.

Furthermore, to minimize unnecessary exploration in less promising regions, we constrain the domain of incrementally added knobs to configurations that have shown to be promising based on historical experience. This constraint of the search space significantly enhances the tuning efficiency by focusing the exploration on more relevant areas.

Discussion. In our work, historical experiences are applied across multiple modules, which establish a robust bridge between past insights and ongoing tuning tasks. When historical experience is unavailable, RABBIT would leverage LLM's ICL capabilities: ❶ For *knob selection* (§VI-A), an external knowledge-empowered LLM evaluates each knob through quantitative scoring and recommends multiple key knobs. ❷ For *knob domain pruning* (§VI-B), the *corrector* leverages the LLM to directly refine the proposals generated by the *proposer*. ❸ For *incremental stage* (§VII-B), RABBIT defines the domain boundary of new knobs using high-quality values from the task initialization phase (§V-A), rather than transferring candidate values from historical experience. Moreover, our RABBIT does not introduce additional latency to user requests. This is because RABBIT is *running in the background*, meaning that the real-time system could respond to user queries immediately using the default knob configurations without waiting for the tuning process to complete.

TABLE I
THE SETTINGS OF DIFFERENT BENCHMARKS.

	TPC-C	SEATS	YCSB	TPC-H
Type	OLTP	OLTP	OLTP	OLAP
Scale Factor (sf)	10	5	20,000	5
Terminals	32	32	32	32
RO Txns	8%	45%	45%	100%

VIII. EXPERIMENTAL EVALUATION

Experimental Environment. RABBIT [56] is implemented in Python 3.10 and the default DBMS instance is MySQL 8.0. Unless otherwise specified, the LLM used in our experiments is OpenAI's GPT-4-Turbo. Experiments are performed on a server running Ubuntu 24.04 equipped with two 28-Core Intel Xeon 6330 processors, 256 GB RAM, 960 GB SSD.

Experimental Benchmarks. We use four benchmarks within the BENCHBASE [57]. Their details are shown in Table I. Specifically, **(1) TPC-C** [58] simulates an online warehouse-centric order processing scenario, which consists of 5 transaction types that operate on 9 tables; **(2) SEATS** [59] simulates an airline ticketing service application using 10 tables and 6 transaction types; **(3) YCSB** [60] is a standard benchmark for cloud systems, which provides various random access key queries on a single 10-column table; **(4) TPC-H** [61] models an analytical application involving global product distribution. It defines 22 query across 8 tables.

Tuning Settings. Following [17], [23], we first eliminate some knobs that hold no potential, e.g., *file_path*, ultimately retaining 70 initial adjustable knobs. We repeat each experiment three times under different random seeds, and report the mean best performance from all runs. The tuning effects are demonstrated by the *transactions per second* and *95th percentile latency* for OLTP and OLAP workloads, respectively. Note, configurations causing the DBMS to crash are penalized by assigning them half of the lowest throughput and double the 95th percentile latency observed so far. Unless otherwise stated, the total number of iterations is set to 100, and the benchmark is TPC-C for the overall experiments by default.

Baselines. We compare RABBIT against four different knob tuners: Gaussian process (GP) [10], [11], sequential model-based algorithm configuration (SMAC) [21], deep deterministic policy gradient (DDPG) [14], and GPTuner [17].

A. Performance Comparison

Fig. 5 illustrates the tuning effects of RABBIT against existing methods. In TPC-C, RABBIT achieves the highest throughput at $1,842.93$, which is $3.80\times$ over the default configuration and outperforms GPTuner $(2.91\times)$ and DDPG $(2.31\times)$. Similarly, for SEATS, RABBIT delivers a remarkable $5.55\times$ improvement, surpassing GPTuner $(4.86\times)$ and DDPG $(2.23\times)$. In YCSB, RABBIT exhibits a $3.45\times$ enhancement over default configuration, demonstrating clear dominance over all baselines. In TPC-H, RABBIT can effectively reduce the latency by up to 51.98%, significantly outperforming its competitors. We observe that RABBIT and GPTuner exhibit significant effects within the first few iterations, whereas the tuning progress of other methods is slow. This is due to the ineffectiveness of other methods in utilizing external knowledge, which hinders the identification of optimal configurations.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

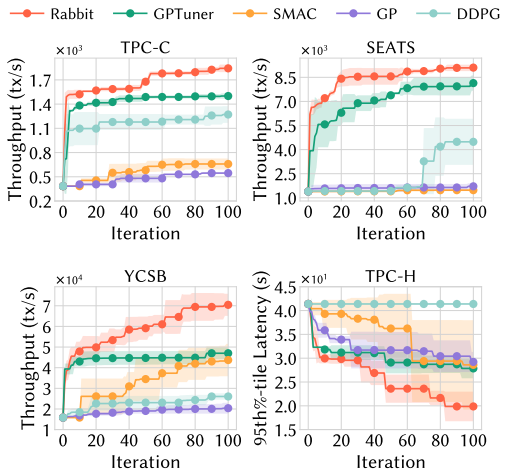


Fig. 5. Performance Comparison across Various Benchmarks over Iterations.

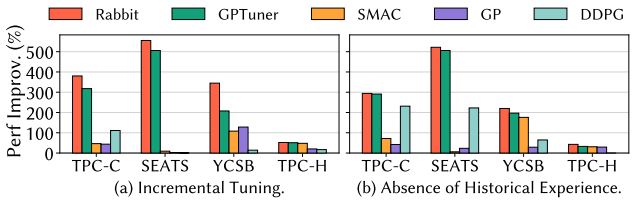


Fig. 6. **End-to-end Tuning Effects under the Following Two Scenarios**: (a) All baselines incrementally incorporate the tunable knobs during the tuning process. (b) In the absence of historical experience, RABBIT adopts the fallback strategy. These two figures share the y-axis coordinates.

Besides, to eliminate the impact of the incremental tuning mode in RABBIT (see §VII-B), all baselines are also enhanced by this mode. As shown in Fig. 6(a), even though other methods are enhanced, RABBIT consistently outperforms them across all benchmarks. This result confirms that while incremental tuning can further improve performance, the sophisticated techniques leveraged by RABBIT complement this mode, working together to achieve optimal results.

Absence of Historical Data. As shown in Fig.6(b), when historical data is unavailable, RABBIT still maintains superior tuning performance, but its superiority is not as significant as the case of leveraging the historical experience. Specifically, our method still achieves 7.8–148.45% higher throughput in YCSB and 15.0–42.8% lower latency in TPC-H compared to the existing methods. These results validate the effectiveness of RABBIT in scenarios lacking prior historical data.

Varying Database Sizes. We analyze the performance comparison with varying scale factors ($sf \in \{10, 100, 500\}$) of TPC-C. As shown in Fig. 7, RABBIT consistently outperforms its competitors. Specifically, at $sf = 100$, it achieves a $1,640.88$ with an improvement of $3.28\times$. This performance surpasses that of GPTuner ($2.98\times$), SMAC ($0.53\times$), GP ($0.48\times$), and DDPG ($1.66\times$). At $sf = 500$, RABBIT achieves $1,659.07$ ($3.32\times$), compared to GPTuner ($3.10\times$), SMAC ($0.50\times$), GP ($0.49\times$), and DDPG ($2.18\times$). Furthermore, RABBIT achieves optimal performance with fewer iterations. For example, at $sf = 100$ and $sf = 500$, it surpasses the best performance of other methods by the 8-th iteration.

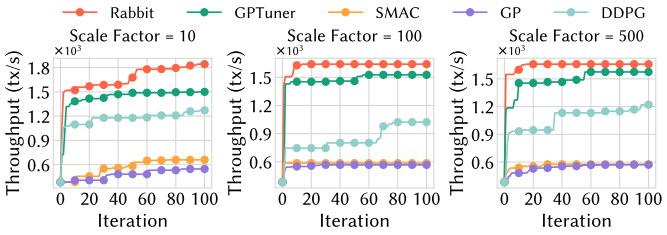


Fig. 7. Performance Comparison on TPC-C with Varying Scale Factors.

B. Ablation Study

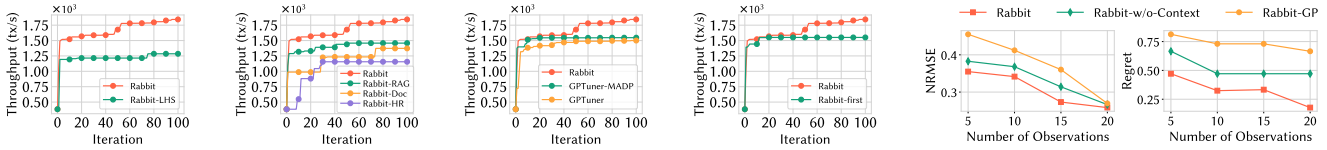
Effect of Task Initialization. To evaluate the effectiveness of the zero-shot initialization strategy in §V-A, we compare it with Latin Hypercube Sampling (LHS), denoted as RABBIT-LHS. As shown in Fig. 8(a), RABBIT achieves a 43.44% improvement over RABBIT-LHS. This is because LHS often selects non-informative regions in a broad initial space, leading to DBMS failures and invalid samples. which hinder early-stage performance boosts. Additionally, configurations generated by LLM sampling exhibit lower correlation, as indicated by a generalized variance close to 0, reflecting greater diversity in the sampled configurations. This diversity facilitates better characterization of the task. Due to the space limitation, the generalized variance is discussed here and not included in the figure, which focuses on end-to-end performance.

Effect of Knowledge Utilization Strategies. We evaluate the impact of different static knowledge utilization strategies while maintaining experience. Specifically, RABBIT-RAG uses the RAG mechanism without the knowledge graph. RABBIT-Doc directly prompts with raw documentation, and RABBIT-HR applies heuristics based on knob relationships from the knowledge graph. As shown in Fig.8(b), RABBIT ($3.80\times$) outperforms all other methods, demonstrating the advantage over leveraging the knowledge base. RABBIT-RAG ($2.80\times$) and RABBIT-Doc ($2.58\times$) show a certain degree of performance degradation, while RABBIT-HR ($2.01\times$) performs the worst, indicating that heuristic methods alone are insufficient.

Effect of MADP. We integrate our MADP technology into GPTuner to create GPTuner-MADP and compare its performance against GPTuner and RABBIT. As shown in Fig. 8(c), GPTuner-MADP achieves better early-stage performance, reaching a throughput of $1,546.68$ by the 18-th iteration, compared to GPTuner's $1,500.14$ after 64 iterations. This improvement is attributed to MADP's ability to alleviate hallucination issues to some extent, enabling more efficient early exploration by refining the search space. However, GPTuner-MADP struggles to match RABBIT in subsequent iterations, as RABBIT incorporates more advanced techniques specifically designed to address hallucination and leverage historical experience for incremental optimization.

Effect of Incremental Configuration Optimization. We compare RABBIT with the complete execution of its first stage, denoted as RABBIT-first. As shown in Fig. 8(d), both methods achieve rapid performance improvements due to the efficient exploration within a compact and effective search space in the first stage. However, RABBIT outperforms RABBIT-first by

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.



(a) Effect of Task Init. (b) Effect of Knowledge. (c) Effect of MADP. (d) Effect of Incre. (e) Effect of Surrogate Model.

Fig. 8. Ablation Study.

TABLE II

OVERHEAD ANALYSIS. TO MEASURE COST-EFFECTIVENESS, THE FOLLOWING METRICS ARE INTRODUCED: (1) **SPPED** IS THE PERFORMANCE GAIN PER MINUTE. (2) **# ITER** & **TIME**: ALL METHODS ARE EARLY TERMINATED WHEN THE TARGET PERFORMANCE (TPC-C (1,500 TPS), SEATS (7,900 TPS), YCSB (47,000 TPS), TPC-H (30S)) ACHIEVED, AND THE ITERATIONS AND TUNING TIME IS RECORDED AS THESE TWO METRICS.

Benchmark	Method	Execution Time							Cost-effectiveness				Memory Usage (MB)			
		Transfer	Knob Selection	Domain Pruning	Rec. (1st / 2nd)	WR	WR (%)	Total	Improv.	Speed	# Iter	Time	Transfer	Knob Selection	Domain Pruning	Rec.
TPC-C	Rabbit	152.4	39.2	138.4	539.4 / 87.4	25,435.6	96.38%	26,392.4	**380.26%** (↑)	**0.86%**	**4**	1,491.2	507.7	626.5	464.2	645.0
	GPTuner	–	497.4	1,762.1	36.2	25,446.6	91.71%	27,742.4	290.93% (↑)	0.63%	97	26,977.9	–	206.5	251.3	212.6
	GP	–	–	–	67.3	25,433.7	99.74%	25,501.0	71.57% (↑)	0.17%	×	×	–	–	–	474.4
	SMAC	–	–	–	21.9	25,412.7	99.91%	25,434.6	42.19% (↑)	0.10%	×	×	–	–	–	150.9
	DDPG	–	–	–	3.06	25,484.8	99.99%	25,487.9	231.26% (↑)	0.54%	×	×	–	–	–	564.7
SEATS	Rabbit	128.4	39.9	147.3	537.2 / 85.8	25,509.9	96.45%	26,448.5	**555.12%** (↑)	**1.26%**	**18**	5,447.8	517.6	638.7	565.6	670.8
	GPTuner	–	548.8	2,141.5	13.7	25,551.8	90.36%	28,255.8	485.69% (↑)	1.07%	65	19,307.9	–	206.5	250.1	215.3
	GP	–	–	–	69.6	25,395.9	99.73%	25,465.5	6.05% (↑)	0.01%	×	×	–	–	–	472.7
	SMAC	–	–	–	27.7	25,485.3	99.89%	25,513.0	23.28% (↑)	0.05%	×	×	–	–	–	136.5
	DDPG	–	–	–	2.93	25,506.5	99.99%	25,509.5	222.60% (↑)	0.52%	×	×	–	–	–	564.8
YCSB	Rabbit	188.9	46.3	160.2	530.0 / 62.1	25,360.3	96.25%	26,347.9	**344.78%** (↑)	**0.79%**	**9**	2,995.9	507.3	665.3	554.6	666.6
	GPTuner	–	490.5	1,883.5	21.1	25,385.1	91.41%	27,780.2	196.72% (↑)	0.42%	88	24,731.4	–	205.6	250.4	213.5
	GP	–	–	–	63.5	25,489.6	99.75%	25,553.2	175.96% (↑)	0.41%	×	×	–	–	–	472.6
	SMAC	–	–	–	25.8	25,536.1	99.90%	25,561.9	28.74% (↑)	0.07%	×	×	–	–	–	136.1
	DDPG	–	–	–	3.15	25,402.8	99.99%	25,406.0	64.56% (↑)	0.15%	×	×	–	–	–	563.6
TPC-H	Rabbit	165.3	42.8	109.1	586.6 / 110.5	25,435.7	96.16%	26,450.0	**51.98%** (↓)	**0.12%**	**11**	3,545.3	503.8	650.7	571.3	629.3
	GPTuner	–	1,169.1	2,568.9	30.9	25,347.0	87.07%	29,115.9	32.69% (↓)	0.07%	48	15,919.4	–	210.8	250.8	213.3
	GP	–	–	–	50.3	25,564.5	99.80%	25,614.8	30.90% (↓)	0.07%	99	25,358.7	–	–	–	473.6
	SMAC	–	–	–	36.6	25,412.0	99.85%	25,449.3	29.40% (↓)	0.07%	64	13,746.3	–	–	–	136.7
	DDPG	–	–	–	6.27	25,536.1	99.98%	25,542.4	0.00% (↓)	0.00%	×	×	–	–	–	566.5

TABLE III

COST ANALYSIS. (1) **P. TOKEN**: THE TOTAL NUMBER OF PROMPT TOKENS INPUT INTO THE LLM. (2) **C. TOKEN**: THE TOTAL NUMBER OF COMPLETION TOKENS GENERATED BY THE LLM. (3) **MONEY** IS MEASURED ACCORDING TO THE OFFICIAL PRICING METRICS FOR GPT-4-TURBO.

Benchmark	Method	Transfer				Knob Selection				Knob Domain Pruning				Configuration Optimization				Total	
		P. Token (k)	C. Token (k)	Money ($)	Time (s)	P. Token (k)	C. Token (k)	Money ($)	Time (s)	P. Token (k)	C. Token (k)	Money ($)	Time (s)	P. Token (k)	C. Token (k)	Money ($)	Time (s)	Money ($)	Time (s)
TPC-C	Rabbit	1.10	3.79	0.12	80.50	79.01	5.83	0.96	35.26	82.14	12.40	1.19	138.38	328.63	19.94	3.88	534.75	6.17	789.89
	GPTuner	–	–	–	–	17.28	26.66	0.97	497.43	492.66	83.93	7.44	1,762.13	–	–	–	–	8.42	2,259.56
SEATS	Rabbit	1.83	3.93	0.14	74.85	64.12	4.42	0.77	35.44	63.01	9.35	0.91	147.34	555.58	18.74	6.12	531.63	7.94	789.25
	GPTuner	–	–	–	–	17.28	26.78	0.98	548.77	534.53	87.88	7.98	2,141.51	–	–	–	–	8.96	2,690.28
YCSB	Rabbit	1.83	4.83	0.16	96.62	54.76	3.95	0.67	42.19	59.58	8.15	0.84	160.24	607.62	20.49	6.69	523.96	8.36	823.01
	GPTuner	–	–	–	–	17.28	27.00	0.98	490.47	501.37	83.63	7.52	1,883.47	–	–	–	–	8.51	2,373.94
TPC-H	Rabbit	1.76	3.87	0.13	82.37	66.81	4.49	0.80	38.11	55.56	5.88	0.73	109.08	561.08	18.98	6.18	580.23	7.85	809.79
	GPTuner	–	–	–	–	49.18	60.39	2.30	1,169.12	535.18	85.59	7.92	2,568.91	–	–	–	–	10.22	3,738.03

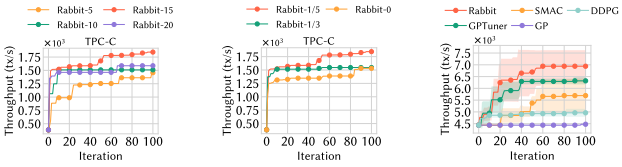
18.86%, benefiting from the incremental stage, which avoids the local optima limitations of the smaller search space.

Effect of Surrogate Model. We employ two widely used metrics to evaluate the surrogate model, i.e., NRMSE for prediction accuracy, and normalized regret for selection quality. To validate the effectiveness of in-context learning, we establish RABBIT-w/o-Context, which simulates surrogate model by using only basic instructions and observations, excluding workload and environment information. Furthermore, to demonstrate the advantages of few-shot surrogate model, we also set up RABBIT-GP, which leverages a traditional surrogate model, i.e., Gaussian process. As shown in Fig. 8(e), the original few-shot surrogate model exhibits significant advantages when only a limited number of observations are available. Additionally, in the absence of in-context learning, the performance of the few-shot surrogate model does not show significant accuracy advantages. These experimental results collectively support our design choice of surrogate model.

TABLE IV

THE RESULTS OF RABBIT WITH VARING LLM.

	1-Shot	# Iter	Perf. (1st)	Perf. (2nd)
GPT-4-Turbo	**1,450.18**	20	**1,586.33**	**1,842.93** (3.80× ↑)
GPT-4o-mini	1,419.48	26	1,569.77	1,594.25 (3.15× ↑)
Claude-3-Opus	1,263.00	**15**	1,431.68	1,522.99 (2.97× ↑)
Llama3-70B	935.60	32	1,159.37	1,201.82 (2.13× ↑)
Llama3-8B	501.82	25	1,309.53	1,405.58 (2.66× ↑)



(a) Knob Selection. (b) Knob Extension. (c) PostgreSQL.

Fig. 9. Parameter Sensitivity Analysis and Effect on Another DBMS.

C. Overhead & Cost Analysis

Runtime Overhead. As shown in Table II, under the fixed 100 iterations, RABBIT incurs an additional runtime of 3.55%–3.78% compared to the fastest method DDPG, while RABBIT is approximately 4.88%–9.16% faster than GPTuner. However, the runtime overhead is dominated by *workload*

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

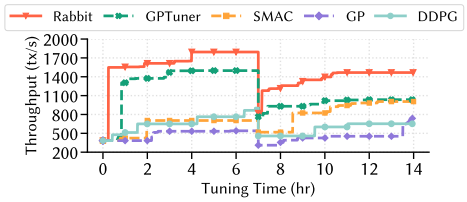


Fig. 10. Adaptability in Dynamically Changing Database Environments.

replay phase, which observes performance feedback on a new configuration for a fixed time period. Thus, reducing iterations is the most effective way to minimize runtime overhead. For cost-effectiveness, compared to the second-best method, RABBIT achieves ❶ a speed improvement of 36.51%, 17.76%, 88.10%, and 71.43% with ❷ a runtime improvement of 94.47%, 71.78%, 87.89%, and 74.21% across the four benchmarks, demonstrating the cost-effectiveness of RABBIT.

Resource Overhead. While RABBIT exhibits relatively higher memory usage than other methods, e.g., an increase of 14.22%, 18.77%, 18.28%, and 11.09% compared to DDPG, the memory usage of RABBIT is around 600MB. Given the substantial memory capacity of modern systems, this overhead is not obvious in large-scale real-world applications.

Cost Analysis. Table III presents a detailed breakdown of ❶ token usage, ❷ associated monetary costs and ❸ time cost related to the use of LLMs. Compared to GPTuner, RABBIT reduces the total monetary cost by 21.1% and time cost by 69.9% on average across all benchmarks, despite introducing additional LLM-invoking steps. Note, RABBIT achieves a significant workload performance improvement at a negligible monetary cost (e.g., 7.94 USD for SEATS). Moreover, these results also reveal that over half of RABBIT's expenses are spent in the recommendation phase, e.g., 62.9% of total monetary cost and 67.7% of total time cost in TPC-C. However, this phase can be terminated early once the desired performance is achieved. In contrast, the entire overhead of GPTuner is incurred during the preparation phase, which cannot be mitigated through early stopping. Consequently, RABBIT has the potential to reduce deployment costs in production environments to less than half of those of GPTuner.

D. Parameter Sensitivity Analysis

Knob Selection. To evaluate the impact of the number of initially selected key knobs $|\boldsymbol{\kappa}_{base}|$ (see §VI-A), we analyze the tuning effectiveness under TPC-C with 5, 10, 15, and 20 knobs. As shown in Fig. 9(a), tuning with $|\boldsymbol{\kappa}_{base}| = 5$ achieves a modest $2.79\times$ improvement, while increasing to $|\boldsymbol{\kappa}_{base}| = 10$ improves performance to $1,510.05$ ($2.94\times$). The best result is obtained with $|\boldsymbol{\kappa}_{base}| = 15$, achieving $1,842.93$ ($3.80\times$). However, further expanding to $|\boldsymbol{\kappa}_{base}| = 20$ reduces performance to $1,587.43$ ($3.14\times$) due to increased search complexity. Therefore, we adopt $|\boldsymbol{\kappa}_{base}| = 15$ as the default configuration to achieve optimal tuning effectiveness.

Knob Extension. We vary the extension ratios between $\frac{1}{3}$, $\frac{1}{5}$, and 0. As shown in Fig. 9(b), we observe that the $\frac{1}{5}$ extension ratio yields the optimal tuning performance. In addition, the $\frac{1}{3}$ ratio shows a 16.08% reduction compared to $\frac{1}{5}$, while no extension results in a 16.91% drop. Moreover, in the absence of knob extension, the upper bound of tuning is constrained due to the omission of critical dependent knobs. Thus, these issues can only be mitigated to some extent during the incremental stage. These findings highlight the importance of knob extension at an appropriate extension ratio.

E. Generalization Analysis

Varying LLM. To evaluate the impact of replacing the LLM in RABBIT, we select 3 closed-source LLMs and 2 open-source LLMs. Table IV demonstrates their effects across the following metrics: ❶ tuning result after the first iteration (1-Shot), ❷ iterations to reach the optimal performance in the first stage (# Iter), ❸ performance at the end of the first stage (Perf. (1st)), and ❹ final tuning performance (Perf. (2nd)).

We observe that closed-source LLMs generally outperform open-source LLMs, owing to their stronger contextual understanding capabilities. Among various LLMs, GPT-4-Turbo consistently achieves the optimal results except # Iter. While Claude-3-Opus achieves the lowest # Iter, its achieved performance is still below the 1-shot result of GPT-4-Turbo.

Varying DBMS. We further evaluate RABBIT on PostgreSQL to assess its adaptability across different DBMSs. As shown in Fig. 9(c), RABBIT achieves the highest performance improvement, with a 56% gain over the default configuration. It outperforms GPTuner (42%), SMAC (28%), GP (1%), and DDPG (12%), highlighting RABBIT's adaptability.

Adaptability in Dynamic Situation. We dynamically adjust the read-write ratio of TPC-C at the 7-hour mark, i.e., making the workload read-heavy for the first 7 hours and write-heavy for the following 7 hours. As shown in Fig. 10, we can observe that when the workload changes abruptly, the existing knob configuration fails to effectively adapt to the newly arriving workload. However, after RABBIT tunes the workload for a short period, its performance is quickly optimized. This is because RABBIT could make full use of the external knowledge to recommend high-quality configurations, giving it a clear advantage over other methods.

IX. CONCLUSION

In this paper, we propose RABBIT, an innovative knob tuner that leverages the retrieval-augmented generation technique to seamlessly integrate multi-source external knowledge. By leveraging an experience-driven knob selection strategy, a multi-agent domain pruning approach, and a dynamic exploration-exploitation mechanism, RABBIT achieves superior tuning performance and significantly outperforms the state-of-the-art techniques. Moreover, RABBIT paves the way for several promising enhancements: (1) A zero-shot tuning paradigm that directly infers optimal configurations from workload and environmental characteristics. (2) Leveraging the similar knowledge-driven idea for tuning other data processing systems, e.g., Spark and Flink. (3) The joint optimization of multiple tuning layers (e.g., indexes, materialized views, and partitioning, etc.) to unlock synergistic benefits.

Acknowledgement. This work is supported by NSFC (62202171, 62461146205) and Fundamental Research Funds for the Central Universities (YBNLTS2024-017).

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. J. Storm, C. Garcia-Arellano, S. S. Lightstone, Y. Diao, and M. Surendra, "Adaptive self-tuning memory in db2," in *VLDB*, 2006, pp. 1081–1092.

[2] D. Narayanan, E. Thereska, and A. Ailamaki, "Continuous resource monitoring for self-predicting dbms," in *MASCOTS*, 2005, pp. 239–248.

[3] W. Tian, P. Martin, and W. Powley, "Techniques for automatically sizing multiple buffer pools in db2," in *CASCON*, 2003, pp. 294–302.

[4] Y. Zhu, J. Liu, M. Guo, Y. Bao, W. Ma, Z. Liu, K. Song, and Y. Yang, "Bestconfig: Tapping the performance potential of systems via automatic configuration tuning," in *SoCC*, 2017, p. 338–350.

[5] S. Huang, Y. Qin, X. Zhang, Y. Tu, Z. Li, and B. Cui, "Survey on performance optimization for database systems," *Science China Information Sciences*, vol. 66, no. 2, p. 121102, 2023.

[6] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, and Z. Li, "An end-to-end automatic cloud database tuning system using deep reinforcement learning," in *SIGMOD*, 2019, p. 415–432.

[7] B. Cai, Y. Liu, C. Zhang, G. Zhang, K. Zhou, L. Liu, C. Li, B. Cheng, J. Yang, and J. Xing, "Hunter: An online cloud database hybrid tuning system for personalized requirements," in *SIGMOD*, 2022, p. 646–659.

[8] G. Li, X. Zhou, S. Li, and B. Gao, "Qtune: A query-aware database tuning system with deep reinforcement learning," *PVLDB*, vol. 12, no. 12, p. 2118–2130, 2019.

[9] X. Zhang, H. Wu, Z. Chang, S. Jin, J. Tan, F. Li, T. Zhang, and B. Cui, "Restune: Resource oriented tuning boosted by meta-learning for cloud databases," in *SIGMOD*, 2021, p. 2102–2114.

[10] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, "Automatic database management system tuning through large-scale machine learning," in *SIGMOD*, 2017, p. 1009–1024.

[11] S. Duan, V. Thummala, and S. Babu, "Tuning database configuration parameters with ituned," *PLVDB*, vol. 2, pp. 1246–1257, 2009.

[12] A. Fekry, L. Carata, T. Pasquier, A. Rice, and A. Hopper, "To tune or not to tune? in search of optimal configurations for data analytics," in *SIGKDD*, 2020, pp. 2494–2504.

[13] M. Kunjir and S. Babu, "Black or white? how to develop an autotuner for memory-based analytics," in *SIGMOD*, 2020, pp. 1667–1683.

[14] D. Van Aken, D. Yang, S. Brillard, A. Fiorino, B. Zhang, C. Bilien, and A. Pavlo, "An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems," *PVLDB*, vol. 14, no. 7, pp. 1241–1253, 2021.

[15] Y. Shen, X. Ren, Y. Lu, H. Jiang, H. Xu, D. Peng, Y. Li, W. Zhang, and B. Cui, "Rover: An online spark sql tuning service via generalized transfer learning," in *SIGKDD*, 2023, p. 4800–4812.

[16] Y. Li, Y. Shen, H. Jiang, T. Bai, W. Zhang, C. Zhang, and B. Cui, "Transfer learning based search space design for hyperparameter tuning," ser. SIGKDD, 2022, p. 967–977.

[17] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, and J. Wang, "Gptuner: A manual-reading database tuning system via gpt-guided bayesian optimization," *arXiv:2311.03157*, 2023.

[18] X. Huang, H. Li, J. Zhang, X. Zhao, Z. Yao, Y. Li, Z. Yu, T. Zhang, H. Chen, and C. Li, "Llmtune: Accelerate database knob tuning with large language models," *arXiv:2404.11581*, 2024.

[19] C. Fan, Z. Pan, W. Sun, C. Yang, and W.-N. Chen, "Latuner: An llm-enhanced database tuning system based on adaptive surrogate model," in *ECML-PKDD*, 2024, pp. 372–388.

[20] I. Trummer, "Db-bert: a database tuning tool that" reads the manual"," in *SIGMOD*, 2022, pp. 190–203.

[21] X. Zhang, Z. Chang, Y. Li, H. Wu, J. Tan, F. Li, and B. Cui, "Facilitating database tuning with hyper-parameter optimization: a comprehensive experimental evaluation," *PVLDB*, vol. 15, no. 9, pp. 1808–1821, 2022.

[22] T. Liu, N. Astorga, N. Seedat, and M. van der Schaar, "Large language models to enhance bayesian optimization," in *ICLR*, 2024.

[23] K. Kanellis, C. Ding, B. Kroth, A. Müller, C. Curino, and S. Venkataraman, "Llamatune: Sample-efficient dbms configuration tuning," *PVLDB*, vol. 15, no. 11, p. 2953–2965, 2022.

[24] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *ICLR*, 2016.

[25] X. Zhang, H. Wu, Y. Li, Z. Tang, J. Tan, F. Li, and B. Cui, "An efficient transfer learning based configuration adviser for database tuning," *PVLDB*, vol. 17, no. 3, pp. 539–552, 2023.

[26] X. Zhao, X. Zhou, and G. Li, "Automatic database knob tuning: A survey," *TKDE*, vol. 35, no. 12, pp. 12 470–12 490, 2023.

[27] Y. Li, H. Li, P. Zhao, J. Zhang, X. Zhang, T. Ji, L. Sun, C. Li, and H. Chen, "Is large language model good at database knob tuning? A comprehensive experimental evaluation," *arXiv:2408.02213*, 2024.

[28] L. Chen, R. Chen, C. Yang, Y. Han, R. Zhang, X. Zhou, P. Jin, and W. Qian, "Workload-aware log-structured merge key-value store for nvm-ssd hybrid storage," in *ICDE*, 2023, pp. 2207–2219.

[29] Y. Han, H. Wang, L. Chen, Y. Dong, X. Chen, B. Yu, C. Yang, and W. Qian, "Bytecard: Enhancing bytedance's data warehouse with learned cardinality estimation," in *SIGMOD*, 2024, pp. 41–54.

[30] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *ICML*, 2023, pp. 15 696–15 707.

[31] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the AI ocean: A survey on hallucination in large language models," *arXiv:2309.01219*, 2023.

[32] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," in *ICLR*, 2020.

[33] J. Wu, C. Chang, T. Yu, Z. He, J. Wang, Y. Hou, and J. J. McAuley, "Coral: Collaborative retrieval-augmented large language models improve long-tail recommendation," in *SIGKDD*, 2024, pp. 3391–3401.

[34] X. Wang, Q. Yang, Y. Qiu, J. Liang, Q. He, Z. Gu, Y. Xiao, and W. Wang, "Knowledgpt: Enhancing large language models with retrieval and storage access on knowledge bases," *arXiv:2308.11761*, 2023.

[35] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv:2310.11511*, 2023.

[36] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv:2404.16130*, 2024.

[37] X. He, X. Bresson, T. Laurent, A. Perold, Y. LeCun, and B. Hooi, "Harnessing explanations: Llm-to-lm interpreter for enhanced text-attributed graph representation learning," *arXiv:2305.19523*, 2023.

[38] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, and J. Han, "Large language models on graphs: A comprehensive survey," *TKDE*, 2024.

[39] V. A. Traag, L. Waltman, and N. J. Van Eck, "From louvain to leiden: guaranteeing well-connected communities," *Scientific reports*, vol. 9, no. 1, pp. 1–12, 2019.

[40] V. Perrone, H. Shen, M. W. Seeger, C. Archambeau, and R. Jenatton, "Learning search spaces for bayesian optimization: Another view of hyperparameter transfer learning," *NeurIPS*, vol. 32, 2019.

[41] X. Zhang, H. Wu, Z. Chang, S. Jin, J. Tan, F. Li, T. Zhang, and B. Cui, "Restune: Resource oriented tuning boosted by meta-learning for cloud databases," in *SIGMOD*, 2021, pp. 2102–2114.

[42] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *NIPS*, 2017, pp. 4765–4774.

[43] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, 2020.

[44] K. Kanellis, R. Alagappan, and S. Venkataraman, "Too many knobs to tune? towards faster database tuning by pre-selecting important knobs," in *HotStorage*, 2020.

[45] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *ICLR*, 2023.

[46] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for A multi-agent collaborative framework," in *ICLR*, 2024.

[47] J. Chen and J. Mueller, "Quantifying uncertainty in answers from any language model and enhancing their trustworthiness," *arXiv*, vol. 2308.16175, 2023.

[48] Z. Lin, S. Trivedi, and J. Sun, "Generating with confidence: Uncertainty quantification for black-box large language models," *ArXiv*, vol. 2305.19187, 2023.

[49] S. H. Tanneru, C. Agarwal, and H. Lakkaraju, "Quantifying uncertainty in natural language explanations of large language models," *arXiv:2311.03533*, 2023.

[50] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, and B. Hooi, "Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms," *arXiv*, vol. 2306.13063, 2023.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.

[51] H. Zhou, X. Wan, L. Proleev, D. Mincu, J. Chen, K. A. Heller, and S. Roy, "Batch calibration: Rethinking calibration for in-context learning and prompt engineering," in *ICLR*, 2024.
[52] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," *arXiv:2104.08786*, 2021.
[53] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," *arXiv:2102.09690*, 2021.
[54] Z. Pan, Y. Wang, Y. Zhang, S. B. Yang, Y. Cheng, P. Chen, C. Guo, Q. Wen, X. Tian, Y. Dou *et al.*, "Magicscaler: Uncertainty-aware, predictive autoscaling," *PVLDB*, vol. 16, no. 12, pp. 3808–3821, 2023.
[55] R. S. Sutton, "Reinforcement learning: An introduction," *A Bradford Book*, 2018.
[56] "Rabbit source code and supplementary materials," 2024. [Online]. Available: https://github.com/dase955/Rabbit.git
[57] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudré-Mauroux, "Oltp-bench: An extensible testbed for benchmarking relational databases," *VLDB*, vol. 7, no. 4, pp. 277–288, 2013.
[58] TPC-C, "TPC-C benchmark," http://www.tpc.org/tpcc/, 1992.
[59] M. Stonebraker and A. Pavlo, "The seats airline ticketing systems benchmark," http://hstore.cs.brown.edu/projects/seats, 2021.
[60] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *SoCC*, 2010, pp. 143–154.
[61] TPC-H, "TPC-H benchmark," http://www.tpc.org/tpch/, 1999.

Authorized licensed use limited to: Zhejiang University. Downloaded on October 28,2026 at 04:04:30 UTC from IEEE Xplore. Restrictions apply.